D A T A B A S E D E S I G N S P E C I F I C A T I O N

Diseño Detallado de la Base de Datos

Módulo de Integración con el ERP — PostgreSQL 16

Proyecto: proyecto_cuentasxcobrar

Documento: DDB-INT-001 · Versión 1.0

Módulo: Integración ERP (Etapa 1)

Fecha: 11 de noviembre de 2026

Base de datos: PostgreSQL 16

Estado: Borrador para revisión

Índice de Contenido

Resumen Ejecutivo.....

Resumen Ejecutivo.....1

1. Introducción.....1

1.1 Propósito y alcance del documento.....1

1.2 Relación con los requisitos y la arquitectura.....2

1.3 Convenciones del documento.....2

2. Decisiones de Diseño de la Base de Datos.....2

2.1 Gestor y versión.....2

2.2 Organización por esquemas (un esquema por módulo).....3

2.3 Convenciones de nomenclatura.....3

2.4 Auditoría, inmutabilidad y borrado lógico.....4

2.5 Estrategia de claves e idempotencia.....4

2.6 Tipos de datos estándar.....5

3. Modelo Conceptual.....5

3.1 Subdominios y entidades.....5

3.2 Mapa de relaciones.....5

4. Modelo Lógico — Diccionario de Tablas.....6

4.1 Esquema integracion — control de carga.....6

Tabla: integracion.conexion_erp.....6

Tabla: integracion.parametro_integracion.....7

Tabla: integracion.mapeo_campo_erp.....7

Tabla: integracion.carga.....7

Tabla: integracion.carga_detalle.....8

Tabla: integracion.error_carga.....8

Tabla: integracion.bitacora_integracion (particionada).....8

4.2 Esquema cartera — entidades cargadas desde el ERP.....9

Tabla: cartera.condicion_pago....9
Tabla: cartera.cliente....9
Tabla: cartera.factura....10
Tabla: cartera.servicio_facturado....10
Tabla: cartera.pago....10
Tabla: cartera.aplicacion_pago....11
Tabla: cartera.snapshot_cartera (particionada)....11
5. Modelo Físico — DDL....12
5.1 Esquemas, roles y permisos....12
5.2 Tablas del esquema integracion....12
5.3 Tablas del esquema cartera....12
6. Estrategias Operativas....12
6.1 Idempotencia y deduplicación....12
6.2 Transacción por carga y manejo de errores....13
6.3 Particionamiento de tablas históricas....13
6.4 Retención y purga....13
6.5 Vistas de conveniencia....14
7. Seguridad a Nivel de Base de Datos....14
7.1 Roles y mínimo privilegio....14
7.2 Protección de credenciales del ERP....14
7.3 Auditoría de acceso....15
8. Migraciones y Versionado del Esquema....15
8.1 Herramienta de migraciones....15
8.2 Convenciones de las migraciones....15
8.3 Compatibilidad hacia atrás....16
9. Volumetría Estimada y Trazabilidad....16
9.1 Volumetría estimada y crecimiento....16
9.2 Trazabilidad de requisitos a tablas....17

Nota: este índice se genera mediante códigos de campo. Tras editar el documento, haga clic derecho sobre el índice y seleccione “Actualizar campos” para refrescar los números de página.

Resumen Ejecutivo

Este documento define el diseño detallado de la base de datos del módulo de Integración con el ERP del Sistema de Gestión de Cuentas por Cobrar. El módulo es el primero en construirse (Etapa 1 del plan) porque abastece de datos financieros al resto del sistema: extrae del sistema contable las facturas, su detalle a nivel de servicios, los pagos y aplicaciones, y los maestros de clientes y condiciones de pago, y los persiste en PostgreSQL de forma idempotente y trazable. El presente diseño cubre el modelo conceptual, el modelo lógico (diccionario de tablas) y el modelo físico (DDL), así como las estrategias operativas de idempotencia, particionamiento, seguridad y migraciones que regirán la implementación.

El esquema se organiza por subdominios sobre PostgreSQL 16, con un esquema propio para el control de la carga (integracion) y otro para los datos de cartera que la integración alimenta (cartera). Todas las tablas transaccionales incorporan columnas de auditoría (created_at, updated_at) y se gestionan con borrado lógico (activo, fecha_baja), en cumplimiento del requisito de inmutabilidad operativa (RNF-SEG-06) del SRS. La idempotencia se garantiza mediante restricciones UNIQUE sobre las claves naturales de negocio y la operación UPSERT (INSERT … ON CONFLICT) de PostgreSQL, de modo que las re-ejecuciones de una carga nunca duplican ni corrompen la cartera.

1. Introducción

1.1 Propósito y alcance del documento

El propósito de este documento es proporcionar al equipo de desarrollo la especificación completa de la base de datos del módulo de Integración, en el nivel de detalle necesario para su implementación directa: esquemas, tablas, columnas con sus tipos y restricciones, índices, relaciones, claves de idempotencia y sentencias DDL listas para ejecutar. El alcance se circunscribe al módulo de Integración y a las entidades del subdominio de cartera que dicho módulo carga desde el ERP (clientes, condiciones de pago, facturas, servicios facturados, pagos y aplicaciones de pago). Los módulos de Cobranza, Asignación, Reportes y Seguridad tendrán sus propias especificaciones de datos en documentos posteriores, pero compartirán el mismo gestor, las mismas convenciones y el principio de un esquema por módulo.

1.2 Relación con los requisitos y la arquitectura

Este diseño materializa los requisitos funcionales RF-INT-01 a RF-INT-10 de la Especificación de Requisitos (SRS v1.0), en particular la extracción a nivel de servicios (RF-INT-02), la deduplicación e idempotencia (RF-INT-03), la validación de calidad de datos

(RF-INT-04), la bitácora de cargas (RF-INT-05) y el histórico de cartera (RF-INT-09). Asimismo, satisface los requisitos no funcionales de auditoría (RNF-SEG-03), inmutabilidad operativa (RNF-SEG-06), respaldo (RNF-DIS-03) y crecimiento sostenido (RNF-PER-05). La arquitectura técnica companion define a PHP con Laravel (Eloquent + Query Builder) como capa de acceso; este documento asume esa pila y entrega el DDL que las migraciones de Laravel gestionarán.

1.3 Convenciones del documento

Las sentencias SQL se presentan en bloques de código monoespaciado y son válidas para PostgreSQL 16. Los nombres de esquema y tabla se escriben siempre en minúsculas y en snake_case. En el diccionario, la columna «Clave» indica PK (clave primaria), FK (clave foránea), UQ (restricción única) o NN (no nulo). Los importes se representan con NUMERIC(14,2) salvo los casos que requieran mayor precisión. Las marcas de tiempo usan TIMESTAMPTZ para almacenarse en UTC y presentarse en la zona horaria de la organización.

2. Decisiones de Diseño de la Base de Datos

2.1 Gestor y versión

El sistema utiliza PostgreSQL 16 como único gestor de base de datos relacional. La elección responde a su robustez transaccional, su soporte nativo de particionamiento declarativo, tipos JSONB para contexto de errores y bitácoras, restricciones UNIQUE para idempotencia y la operación UPSERT (ON CONFLICT) que simplifica la deduplicación. PostgreSQL es además el gestor recomendado por la pila PHP/Laravel del documento de arquitectura, con controladores maduros y bien probados.

2.2 Organización por esquemas (un esquema por módulo)

Siguiendo el estilo de monolito modular del SRS, la base de datos se organiza en un esquema por módulo, de modo que cada módulo es propietario de sus tablas y solo las expone a los demás mediante contratos explícitos. Para el módulo de Integración se definen dos esquemas: integracion, con las tablas de control de carga (carga, detalle, errores, bitácora, parámetros y mapeo de campos); y cartera, con las entidades de negocio que la integración carga desde el ERP (cliente, condicion_pago, factura, servicio_facturado, pago, aplicacion_pago, snapshot_cartera). Aunque cartera será enriquecido por módulos posteriores (Cobranza, Asignación), su creación y alimentación inicial corresponden a la Etapa 1.

Tabla 1. Esquemas definidos en la Etapa 1

Esquema	Responsabilidad	Propietario (módulo)
integracion	Control de cargas, bitácora, cola de errores, parámetros y mapeo de campos del ERP.	Integración ERP
cartera	Maestros y transaccional de la cartera cargada desde el ERP: clientes, condiciones de pago, facturas, servicios, pagos y aplicaciones.	Integración ERP (carga) / Cartera (lectura y enriquecimiento)

2.3 Convenciones de nomenclatura

- Todos los identificadores en minúsculas y snake_case (cliente, fecha_vencimiento, saldo_pendiente).
- Nombres de tabla en singular (factura, no facturas), salvo las relaciones N:M que usan el verbo que las describe (aplicacion_pago).
- Claves primarias: columna id de tipo BIGSERIAL (BIGINT autogenerated), salvo tablas particionadas que la combinan con la columna de partición.
- Claves foráneas: nombre de la entidad referenciada con sufijo _id (cliente_id, factura_id, carga_id).
- Columnas booleanas de estado con valor por defecto explícito (activo DEFAULT TRUE).
- Marcas de auditoría: created_at y updated_at de tipo TIMESTAMPTZ con DEFAULT now().
- Columnas monetarias: NUMERIC(14,2); cantidades: NUMERIC(12,3); códigos cortos: VARCHAR con longitud definida.

2.4 Auditoría, inmutabilidad y borrado lógico

Para cumplir el requisito de inmutabilidad operativa (RNF-SEG-06), ninguna tabla transaccional permite el borrado físico desde la aplicación. Los maestros y las cuentas usan borrado lógico mediante una columna activo BOOLEAN y una marca fecha_baja TIMESTAMPTZ que se establece al desactivar el registro. Las tablas de control de carga (bitacora_integracion, error_carga) son de solo inserción: ningún rol de aplicación las modifica ni elimina, y su purga se realiza solo por un procedimiento administrativo programado, nunca desde la interfaz. Adicionalmente, toda tabla de negocio incluye created_at y updated_at, y las que admiten modificación incluyen updated_by para identificar al usuario o al proceso que la modificó.

2.5 Estrategia de claves e idempotencia

Cada tabla dispone de una clave primaria surrogate (id BIGSERIAL) para referencias internas eficientes y estables, más una o varias restricciones UNIQUE sobre la clave natural de negocio que identifica de forma única al registro en el ERP. Esta clave natural es la base de la idempotencia: la operación UPSERT (INSERT … ON CONFLICT DO UPDATE) inserta el registro si no existe o lo actualiza si ya estaba, de modo que una misma carga ejecutada dos veces produce exactamente los mismos datos. La tabla 2 resume las claves naturales por entidad.

Tabla 2. Claves naturales de idempotencia por entidad

Entidad	Clave natural (UQ)	Rationale
cliente	codigo_erp	El código del cliente es estable y único en el ERP.
condicion_pago	codigo_erp	Identificador estable de la condición en el ERP.
factura	clave_unicidad (serie + número + tipo o id ERP)	Permite re-cargar sin duplicar; reconstruye la clave desde los campos del ERP.
servicio_facturado	clave_unicidad (factura + código de servicio + período)	Identifica unívocamente una línea de servicio dentro de una factura.
pago	identificador_pago	Identificador del pago en el ERP o en el banco.
aplicacion_pago	(pago_id, factura_id)	Una aplicación de un pago a una factura es única.

2.6 Tipos de datos estándar

Tabla 3. Tipos de datos de uso frecuente

Tipo	Uso
BIGSERIAL / BIGINT	Claves primarias y foráneas.
VARCHAR(n)	Códigos, identificadores y textos acotados.
TEXT	Descripciones y mensajes de error sin longitud fija.
DATE	Fechas de emisión, vencimiento, aplicación, carga.
TIMESTAMPTZ	Marcas de auditoría y de eventos (UTC).
NUMERIC(14,2)	Importes monetarios.
NUMERIC(12,3)	Cantidades.
BOOLEAN	Flags de estado (activo, resuelto).
JSONB	Contexto de errores y bitácora; dato original rechazado.

3. Modelo Conceptual

3.1 Subdominios y entidades

El modelo conceptual agrupa las once entidades del módulo de Integración en tres subdominios: maestros (sincronizados desde el ERP y de baja variabilidad), transaccional (facturas, sus servicios, pagos y aplicaciones, de alta variabilidad diaria) y control de carga (bitácora, errores y snapshot histórico). La figura 1 muestra el catálogo de entidades con sus columnas clave y las marcas PK, FK y UQ; el detalle completo de columnas se presenta en el capítulo 4 (diccionario) y el DDL en el capítulo 5.

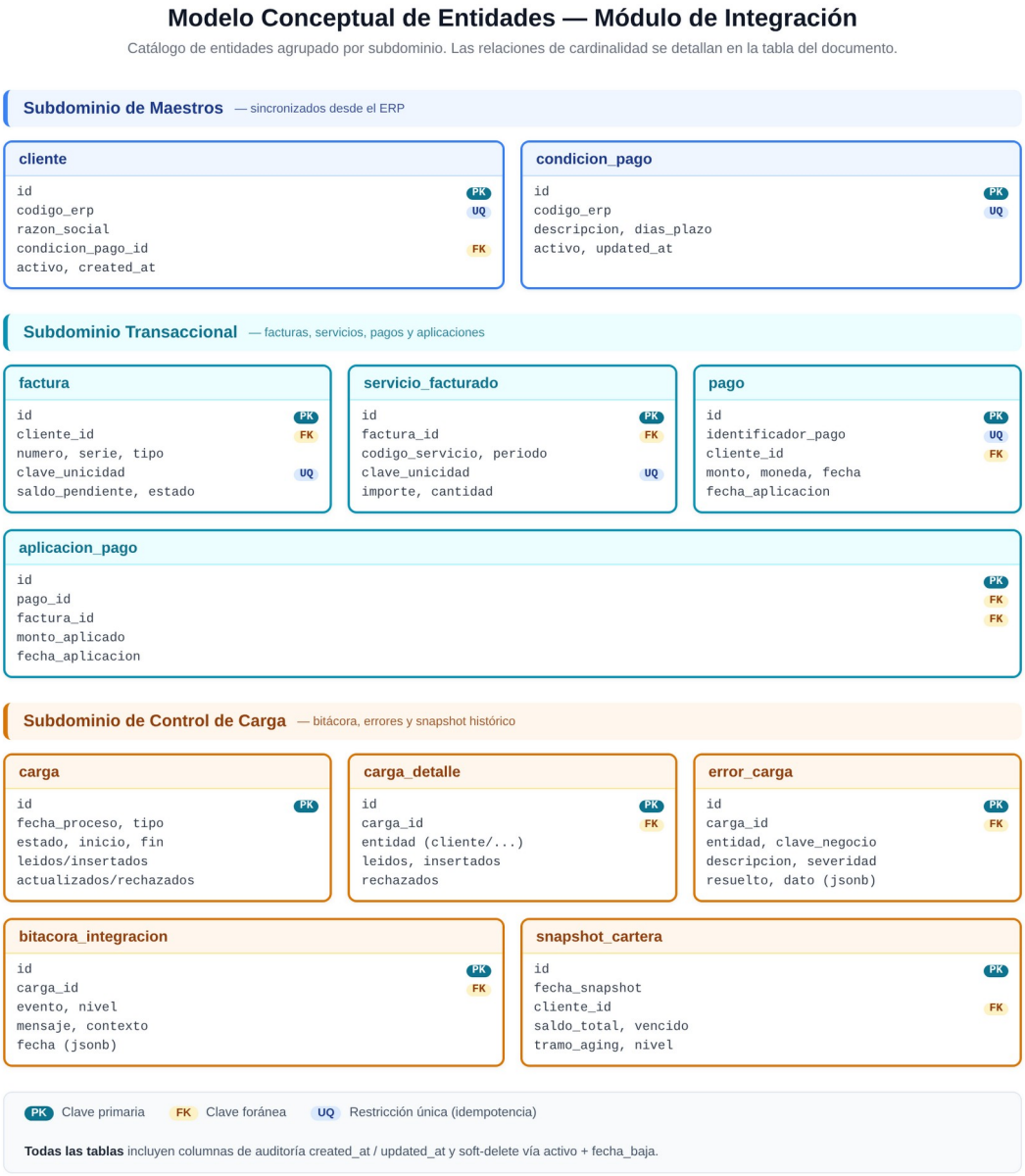


Figura 1. Modelo conceptual de entidades del módulo de Integración

3.2 Mapa de relaciones

La tabla 4 detalla las relaciones entre entidades, su cardinalidad y la regla de integridad que las gobierna. Las relaciones de eliminación se configuran de forma conservadora: solo las dependientes estrictas (detalle de carga, servicios de una factura) usan ON DELETE CASCADE; el resto impide la eliminación de un maestro con dependencias activas.

Tabla 4. Mapa de relaciones del modelo

Relación	Cardinalidad	Regla de integridad
condicion_pago → cliente	1 : N	Un cliente tiene una condición de pago; la condición puede no existir (FK anulable).
cliente → factura	1 : N	Una factura pertenece a un cliente (FK NOT NULL). No se elimina un cliente con facturas.
cliente → pago	1 : N	Un pago pertenece a un cliente (FK NOT NULL).
factura → servicio_facturado	1 : N	Los servicios dependen de la factura (ON DELETE CASCADE).
factura → aplicacion_pago	1 : N	Una factura recibe N aplicaciones de pago.
pago → aplicacion_pago	1 : N	Un pago se aplica a N facturas (ON DELETE CASCADE sobre el pago).
carga → carga_detalle	1 : N	Cada carga tiene un detalle por entidad (UQ carga_id+entidad).
carga → error_carga	1 : N	Los errores se asocian a la carga (ON DELETE CASCADE).
carga → bitacora_integracion	1 : N	Eventos de la carga (ON DELETE CASCADE).
carga → factura / pago	1 : N	Trazabilidad: la factura y el pago registran la carga que los creó o actualizó (carga_id).
cliente → snapshot_cartera	1 : N	El snapshot histórico agrega al cliente por día.

4. Modelo Lógico — Diccionario de Tablas

Este capítulo documenta cada tabla con su descripción y el detalle de columnas. La columna «Clave» usa PK (primaria), FK (foránea), UQ (única), NN (no nulo) y DV (valor por defecto). Las marcas de auditoría created_at, updated_at se omite listarlas en cada tabla por brevedad, pero están presentes en todas las tablas de negocio y de control conforme a la sección 2.4.

4.1 Esquema integracion — control de carga

Tabla: integracion.conexion_erp

Configura las conexiones disponibles al ERP (una activa a la vez); el mecanismo implementado es el API REST del ERP. Las credenciales nunca se almacenan en claro: credenciales_ref referencia el secreto cifrado (token o par cliente-servidor OAuth2 del API REST) gestionado por la capa de aplicación.

Tabla 5. integracion.conexion_erp

Columna	Tipo	Clave	Descripción
id	BIGSERIAL	PK	Identificador interno.
nombre	VARCHAR(80)	NN	Nombre descriptivo de la conexión.
tipo	VARCHAR(16)	NN	api (REST, mecanismo implementado) | bd | archivos (reservados por el patrón puerto-adaptador).
endpoint	VARCHAR(255)		URL base del API REST del ERP.
credenciales_ref	VARCHAR(120)	NN	Referencia al secreto cifrado, no el valor.
activo	BOOLEAN	NN, DV TRUE	Indica si es la conexión vigente.

Tabla: integracion.parametro_integracion

Parámetros configurables del proceso de integración: ventana de carga, umbrales de alerta, directorios de intercambio. Editables por el administrador sin requerir cambios de código.

Tabla 6. integracion.parametro_integracion

Columna	Tipo	Clave	Descripción
id	BIGSERIAL	PK	Identificador interno.
clave	VARCHAR(60)	UQ, NN	Nombre del parámetro (ej. ventana_carga_inicio).
valor	TEXT	NN	Valor del parámetro.
descripcion	VARCHAR(200)		Descripción para el administrador.
editable	BOOLEAN	NN, DV TRUE	Si puede modificarse desde la interfaz.
updated_by	BIGINT		Usuario que modificó el parámetro.

Tabla: integracion.mapeo_campo_erp

Diccionario de mapeo entre los campos físicos del ERP y los campos locales, con la transformación a aplicar. Permite reconfigurar la integración cuando el ERP cambia sin tocar el código del conector.

Tabla 7. integracion.mapeo_campo_erp

Columna	Tipo	Clave	Descripción
id	BIGSERIAL	PK	Identificador interno.
entidad_erp	VARCHAR(40)	UQ, NN	Entidad origen en el ERP (cliente, factura, …).
campo_erp	VARCHAR(60)	UQ, NN	Campo físico en el ERP.
campo_local	VARCHAR(60)	NN	Campo destino en el modelo local.
transformacion	VARCHAR(120)		Función de transformación (fecha_iso, decimal_2, …).
obligatorio	BOOLEAN	NN, DV FALSE	Si el campo es obligatorio para la carga.

Tabla: integracion.carga

Registro de cada ejecución de carga (batch nocturno, API dirigida o reproceso). Es la cabecera de la bitácora de integración y el origen de la trazabilidad de los datos cargados.

Tabla 8. integracion.carga

Columna	Tipo	Clave	Descripción
id	BIGSERIAL	PK	Identificador de la carga.
fecha_proceso	DATE	NN	Día whosecartera se procesa.
origen	VARCHAR(40)	NN	batch_nocturno | api_dirigida | reproceso.
tipo	VARCHAR(16)	NN	completa | incremental.
estado	VARCHAR(16)	NN	en_curso | ok | ok_errores | fallida.
inicio	TIMESTAMPTZ	NN, DV now()	Inicio de la ejecución.
fin	TIMESTAMPTZ		Fin de la ejecución.
duracion_ms	INTEGER		Duración en milisegundos.
registros_leidos	INTEGER	NN, DV 0	Total leídos del ERP.
registros_insertados	INTEGER	NN, DV 0	Nuevos insertados.
registros_actualizados	INTEGER	NN, DV 0	Actualizados por UPSERT.
registros_rechazados	INTEGER	NN, DV 0	Rechazados por validación.
usuario_reproceso	BIGINT		Usuario que disparó el reproceso, si aplica.

Tabla: integracion.carga_detalle

Desglose por entidad de los contadores de una carga (clientes, facturas, servicios, pagos). Permite ver el resultado de la carga con granularidad de entidad.

Tabla 9. integracion.carga_detalle

Columna	Tipo	Clave	Descripción
id	BIGSERIAL	PK	Identificador interno.
carga_id	BIGINT	FK, NN	Referencia a integracion.carga.
entidad	VARCHAR(30)	UQ, NN	cliente | factura | servicio | pago.
registros_leidos	INTEGER	NN, DV 0	Leídos de esta entidad.
registros_insertados	INTEGER	NN, DV 0	Insertados de esta entidad.
registros_actualizados	INTEGER	NN, DV 0	Actualizados de esta entidad.
registros_rechazados	INTEGER	NN, DV 0	Rechazados de esta entidad.

Tabla: integracion.error_carga

Cola de errores de la carga: cada registro rechazado por validación queda aquí con su causa, el dato original (JSONB) y un indicador de resuelto. El administrador técnica analiza y corrige, y marca el error como resuelto una vez reprocesado.

Tabla 10. integracion.error_carga

Columna	Tipo	Clave	Descripción
id	BIGSERIAL	PK	Identificador del error.
carga_id	BIGINT	FK, NN	Carga en la que se produjo.
entidad	VARCHAR(30)	NN	Entidad del registro fallido.
clave_negocio	VARCHAR(120)		Identificador de negocio del registro.
campo	VARCHAR(60)		Campo que originó el error, si aplica.
descripcion	TEXT	NN	Descripción del error en lenguaje claro.
severidad	VARCHAR(10)	NN	baja | media | alta.
dato_original	JSONB		Volcado del registro que falló, para reproceso.
resuelto	BOOLEAN	NN, DV FALSE	Si el error fue corregido y reprocesado.
fecha	TIMESTAMPTZ	NN, DV now()	Momento del error.

Tabla: integracion.bitacora_integracion (particionada)

Bitácora de eventos de la integración: inicio y fin de cada paso, advertencias y errores. Se particiona por mes (sección 6.3) por su alto volumen y carácter histórico; es de solo inserción.

Tabla 11. integracion.bitacora_integracion

Columna	Tipo	Clave	Descripción
id	BIGSERIAL	PK	Identificador (parte de la PK particionada).
carga_id	BIGINT	FK	Carga asociada, si aplica.
evento	VARCHAR(60)	NN	Nombre del evento.
nivel	VARCHAR(8)	NN	INFO | WARN | ERROR.
mensaje	TEXT	NN	Mensaje del evento.
contexto	JSONB		Contexto adicional estructurado.
fecha	TIMESTAMPTZ	PK, NN, DV now()	Momento del evento (clave de partición).

4.2 Esquema cartera — entidades cargadas desde el ERP

Tabla: cartera.condicion_pago

Maestro de condiciones de pago sincronizado desde el ERP. Borrado lógico; el campo dias_plazo alimenta el cálculo de la fecha de vencimiento esperada.

Tabla 12. cartera.condicion_pago

Columna	Tipo	Clave	Descripción
id	BIGSERIAL	PK	Identificador interno.
codigo_erp	VARCHAR(30)	UQ, NN	Código estable de la condición en el ERP.
descripcion	VARCHAR(120)	NN	Descripción legible.
dias_plazo	INTEGER	NN, DV 0	Días de plazo de la condición.
activo	BOOLEAN	NN, DV TRUE	Borrado lógico.
fecha_baja	TIMESTAMPTZ		Momento de la baja lógica.

Tabla: cartera.cliente

Maestro de clientes sincronizado desde el ERP, con los datos de contacto de cobranza que el módulo de Cobranza enriquecerá. La clave codigo_erp garantiza la idempotencia de la sincronización.

Tabla 13. cartera.cliente

Columna	Tipo	Clave	Descripción
id	BIGSERIAL	PK	Identificador interno.
codigo_erp	VARCHAR(30)	UQ, NN	Código único del cliente en el ERP.
razon_social	VARCHAR(180)	NN	Razón social o nombre.

Columna	Tipo	Clave	Descripción
identificacion_fiscal	VARCHAR(40)		RFC/NIT/CUIT u otro.
direccion	VARCHAR(200)		Dirección fiscal/comercial.
condicion_pago_id	BIGINT	FK	Condición de pago asignada.
limite_credito	NUMERIC(14,2)		Límite de crédito del ERP.
estado_credito	VARCHAR(20)	NN, DV 'activo'	Estado de crédito del ERP.
telefono_cobranza	VARCHAR(40)		Teléfono de contacto de cobranza.
email_cobranza	VARCHAR(120)		Correo de contacto de cobranza.
fecha_alta_erp	DATE		Fecha de alta en el ERP.
activo	BOOLEAN	NN, DV TRUE	Borrado lógico.
fecha_baja	TIMESTAMPTZ		Momento de la baja lógica.

Tabla: cartera.factura

Documento por cobrar emitido en el ERP. La columna clave_unicidad (construida a partir de serie, número y tipo o del identificador del ERP) garantiza la idempotencia de la carga. saldo_pendiente y estado_contable se actualizan en cada carga con los pagos aplicados reportados por el ERP.

Tabla 14. cartera.factura

Columna	Tipo	Clave	Descripción
id	BIGSERIAL	PK	Identificador interno.
cliente_id	BIGINT	FK, NN	Cliente de la factura.
numero	VARCHAR(40)	NN	Número de factura.
serie	VARCHAR(20)		Serie o prefijo.
tipo	VARCHAR(20)	NN	Tipo de documento (factura, nota de crédito, …).
clave_unicidad	VARCHAR(80)	UQ, NN	Clave natural de idempotencia.
fecha_emision	DATE	NN	Fecha de emisión.
fecha_vencimiento	DATE	NN	Fecha de vencimiento.
moneda	VARCHAR(3)	NN, DV 'USD'	Código de moneda ISO 4217.
importe_bruto	NUMERIC(14,2)	NN	Importe bruto.
impuestos	NUMERIC(14,2)	NN, DV 0	Impuestos.
total	NUMERIC(14,2)	NN	Total facturado.
saldo_pendiente	NUMERIC(14,2)	NN	Saldo pendiente de pago.
estado_contable	VARCHAR(20)	NN	Estado contable del ERP.

Columna	Tipo	Clave	Descripción
fecha_carga	DATE	NN	Fecha de la carga que la creó o actualizó.
carga_id	BIGINT	FK	Carga que la creó o actualizó.

Tabla: cartera.servicio_facturado

Detalle a nivel de servicios de cada factura: la unidad mínima de análisis y conciliación exigida por el alcance. Cada servicio se identifica unívocamente dentro de su factura por la combinación de código de servicio y período, materializada en clave_unicidad.

Tabla 15. cartera.servicio_facturado

Columna	Tipo	Clave	Descripción
id	BIGSERIAL	PK	Identificador interno.
factura_id	BIGINT	FK, NN	Factura a la que pertenece.
codigo_servicio	VARCHAR(40)	NN	Código del servicio facturado.
descripcion	VARCHAR(200)		Descripción del servicio.
periodo	VARCHAR(20)		Período facturado.
cantidad	NUMERIC(12,3)	NN, DV 1	Cantidad facturada.
precio_unitario	NUMERIC(14,2)	NN	Precio unitario.
importe	NUMERIC(14,2)	NN	Importe del servicio.
centro_costo	VARCHAR(40)		Centro de costo, si aplica.
referencia_contratual	VARCHAR(60)		Referencia contractual.
clave_unicidad	VARCHAR(100)	UQ, NN	Clave natural (factura + servicio + período).

Tabla: cartera.pago

Pago reportado por el ERP. Su identificador_pago garantiza la idempotencia; las aplicaciones a facturas se modelan en la tabla aplicacion_pago.

Tabla 16. cartera.pago

Columna	Tipo	Clave	Descripción
id	BIGSERIAL	PK	Identificador interno.
identificador_pago	VARCHAR(60)	UQ, NN	Identificador del pago en ERP/banco.
cliente_id	BIGINT	FK, NN	Cliente que pagó.
fecha	DATE	NN	Fecha del pago.
monto	NUMERIC(14,2)	NN	Monto del pago.

Columna	Tipo	Clave	Descripción
moneda	VARCHAR(3)	NN, DV 'USD'	Moneda del pago.
metodo	VARCHAR(30)		Método (transferencia, cheque, …).
banco	VARCHAR(60)		Banco origen.
referencia_bancaria	VARCHAR(80)		Referencia bancaria.
fecha_aplicacion	DATE		Fecha de aplicación contable.
fecha_carga	DATE	NN	Fecha de la carga.
carga_id	BIGINT	FK	Carga que lo cargó.

Tabla: cartera.aplicacion_pago

Relación N:M entre pagos y facturas: cada aplicación indica cuánto de un pago se aplicó a una factura. La restricción UNIQUE (pago_id, factura_id) impide duplicar aplicaciones.

Tabla 17. cartera.aplicacion_pago

Columna	Tipo	Clave	Descripción
id	BIGSERIAL	PK	Identificador interno.
pago_id	BIGINT	FK, NN	Pago aplicado.
factura_id	BIGINT	FK, NN	Factura a la que se aplica.
monto_aplicado	NUMERIC(14,2)	NN	Monto aplicado a esa factura.
fecha_aplicacion	DATE	NN	Fecha de aplicación.

Tabla: cartera.snapshot_cartera (particionada)

Fotografía diaria consolidada del saldo y aging de cada cliente, que permite reconstruir indicadores históricos sin recurrir al ERP. Se particiona por mes por su volumen creciente y carácter histórico.

Tabla 18. cartera.snapshot_cartera

Columna	Tipo	Clave	Descripción
id	BIGSERIAL	PK	Identificador (parte de la PK particionada).
fecha_snapshot	DATE	PK, NN	Día del snapshot (clave de partición).
cliente_id	BIGINT	FK, NN	Cliente al que agrega.
saldo_total	NUMERIC(14,2)	NN, DV 0	Saldo total del cliente ese día.
saldo_vencido	NUMERIC(14,2)	NN, DV 0	Saldo vencido ese día.
tramo_aging	VARCHAR(10)	NN	Tramo dominante (corriente, 1-30, …).
nivel_cobranza	VARCHAR(4)	NN	Nivel de cobranza N0-N4.

5. Modelo Físico — DDL

5.1 Esquemas, roles y permisos

La creación comienza por los esquemas y los roles de base de datos, siguiendo el principio de mínimo privilegio. La aplicación web opera con el rol cxc_app, el proceso de carga con cxc_loader, y los reportes y la auditoría con cxc_readonly. La propiedad de los esquemas recae en cxc_owner, rol administrativo sin acceso desde la aplicación.

```
-- Roles de base de datos (mínimo privilegio)
CREATE ROLE cxc_owner   LOGIN;
CREATE ROLE cxc_app     LOGIN;     -- aplicación web (lectura/escritura de
negocio)
CREATE ROLE cxc_loader  LOGIN;     -- proceso de carga batch (escritura
integracion+cartera)
CREATE ROLE cxc_readonly;          -- reportes y auditoría (solo lectura)

-- Esquemas por módulo
CREATE SCHEMA IF NOT EXISTS integracion AUTHORIZATION cxc_owner;
CREATE SCHEMA IF NOT EXISTS cartera     AUTHORIZATION cxc_owner;

-- Permisos base
GRANT USAGE ON SCHEMA integracion, cartera TO cxc_app, cxc_loader,
cxc_readonly;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA cartera, integracion TO
cxc_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA integracion TO
cxc_loader;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA cartera TO cxc_loader;
GRANT SELECT ON ALL TABLES IN SCHEMA integracion, cartera TO cxc_readonly;
ALTER DEFAULT PRIVILEGES IN SCHEMA integracion, cartera
  GRANT SELECT, INSERT, UPDATE ON TABLES TO cxc_app;
```

5.2 Tablas del esquema integracion

```
-- Catálogo de conexiones configuradas al ERP
CREATE TABLE integracion.conexion_erp (
    id               BIGSERIAL PRIMARY KEY,
    nombre           VARCHAR(80)  NOT NULL,
    tipo             VARCHAR(16)  NOT NULL CHECK (tipo IN
('api','bd','archivos')),  -- 'api'=REST (implementado)
    endpoint         VARCHAR(255),                          -- URL base del
API REST del ERP
    credenciales_ref VARCHAR(120) NOT NULL,   -- referencia al secreto
(token/par OAuth2), nunca el valor
    activo           BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at       TIMESTAMPTZ  NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CONSTRAINT conexion_erp_unica_activa UNIQUE (activo) DEFERRABLE INITIALLY
DEFERRED
    FILTER (WHERE activo = TRUE)            -- solo una conexión activa a la
vez
);

-- Parámetros configurables del proceso de integración
CREATE TABLE integracion.parametro_integracion (
    id          BIGSERIAL PRIMARY KEY,
    clave       VARCHAR(60) NOT NULL UNIQUE,
    valor       TEXT        NOT NULL,
```

```
    descripcion VARCHAR(200),
    editable    BOOLEAN     NOT NULL DEFAULT TRUE,
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_by  BIGINT
);

-- Mapeo de campos ERP -> modelo local
CREATE TABLE integracion.mapeo_campo_erp (
    id             BIGSERIAL PRIMARY KEY,
    entidad_erp    VARCHAR(40) NOT NULL,
    campo_erp      VARCHAR(60) NOT NULL,
    campo_local    VARCHAR(60) NOT NULL,
    transformacion VARCHAR(120),
    obligatorio    BOOLEAN     NOT NULL DEFAULT FALSE,
    UNIQUE (entidad_erp, campo_erp)
);

-- Ejecución de una carga (cabecera de la bitácora)
CREATE TABLE integracion.carga (
    id                  BIGSERIAL PRIMARY KEY,
    fecha_proceso       DATE        NOT NULL,
    origen              VARCHAR(40) NOT NULL CHECK (origen IN
('batch_nocturno','api_dirigida','reproceso')),
    tipo                VARCHAR(16) NOT NULL CHECK (tipo IN
('completa','incremental')),
    estado              VARCHAR(16) NOT NULL CHECK (estado IN
('en_curso','ok','ok_errores','fallida')),
    inicio              TIMESTAMPTZ NOT NULL DEFAULT now(),
    fin                 TIMESTAMPTZ,
    duracion_ms         INTEGER,
    registros_leidos    INTEGER     NOT NULL DEFAULT 0,
    registros_insertados INTEGER    NOT NULL DEFAULT 0,
    registros_actualizados INTEGER   NOT NULL DEFAULT 0,
    registros_rechazados INTEGER    NOT NULL DEFAULT 0,
    usuario_reproceso   BIGINT,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_carga_fecha  ON integracion.carga (fecha_proceso DESC);
CREATE INDEX idx_carga_estado ON integracion.carga (estado);

-- Detalle por entidad de una carga
CREATE TABLE integracion.carga_detalle (
    id                  BIGSERIAL PRIMARY KEY,
    carga_id            BIGINT      NOT NULL REFERENCES integracion.carga(id)
ON DELETE CASCADE,
    entidad             VARCHAR(30) NOT NULL,
    registros_leidos    INTEGER     NOT NULL DEFAULT 0,
    registros_insertados INTEGER    NOT NULL DEFAULT 0,
    registros_actualizados INTEGER   NOT NULL DEFAULT 0,
    registros_rechazados INTEGER    NOT NULL DEFAULT 0,
    UNIQUE (carga_id, entidad)
);

-- Cola de errores de la carga
CREATE TABLE integracion.error_carga (
    id            BIGSERIAL PRIMARY KEY,
    carga_id      BIGINT      NOT NULL REFERENCES integracion.carga(id) ON
DELETE CASCADE,
    entidad       VARCHAR(30) NOT NULL,
    clave_negocio VARCHAR(120),
    campo         VARCHAR(60),
    descripcion   TEXT        NOT NULL,
    severidad     VARCHAR(10) NOT NULL CHECK (severidad IN
('baja','media','alta')),
    dato_original JSONB,
```

```
    resuelto      BOOLEAN     NOT NULL DEFAULT FALSE,
    fecha         TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_error_carga_carga    ON integracion.error_carga (carga_id);
CREATE INDEX idx_error_carga_pendientes ON integracion.error_carga (resuelto,
fecha)
    WHERE resuelto = FALSE;

-- Bitácora de integración (particionada por mes)
CREATE TABLE integracion.bitacora_integracion (
    id       BIGSERIAL,
    carga_id BIGINT      REFERENCES integracion.carga(id) ON DELETE CASCADE,
    evento   VARCHAR(60) NOT NULL,
    nivel    VARCHAR(8)  NOT NULL CHECK (nivel IN ('INFO','WARN','ERROR')),
    mensaje  TEXT        NOT NULL,
    contexto JSONB,
    fecha    TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (id, fecha)
) PARTITION BY RANGE (fecha);
```

5.3 Tablas del esquema cartera

```
-- Condición de pago (maestro)
CREATE TABLE cartera.condicion_pago (
    id          BIGSERIAL PRIMARY KEY,
    codigo_erp  VARCHAR(30) NOT NULL UNIQUE,
    descripcion VARCHAR(120) NOT NULL,
    dias_plazo  INTEGER     NOT NULL DEFAULT 0,
    activo      BOOLEAN     NOT NULL DEFAULT TRUE,
    fecha_baja  TIMESTAMPTZ,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Cliente (maestro)
CREATE TABLE cartera.cliente (
    id                   BIGSERIAL PRIMARY KEY,
    codigo_erp           VARCHAR(30)  NOT NULL UNIQUE,
    razon_social         VARCHAR(180) NOT NULL,
    identificacion_fiscal VARCHAR(40),
    direccion            VARCHAR(200),
    condicion_pago_id    BIGINT REFERENCES cartera.condicion_pago(id),
    limite_credito       NUMERIC(14,2),
    estado_credito       VARCHAR(20)  NOT NULL DEFAULT 'activo',
    telefono_cobranza    VARCHAR(40),
    email_cobranza       VARCHAR(120),
    fecha_alta_erp       DATE,
    activo               BOOLEAN      NOT NULL DEFAULT TRUE,
    fecha_baja           TIMESTAMPTZ,
    created_at           TIMESTAMPTZ  NOT NULL DEFAULT now(),
    updated_at           TIMESTAMPTZ  NOT NULL DEFAULT now()
);
CREATE INDEX idx_cliente_razon ON cartera.cliente (razon_social);
CREATE INDEX idx_cliente_activos ON cartera.cliente (activo) WHERE activo =
TRUE;

-- Factura (transaccional)
CREATE TABLE cartera.factura (
    id              BIGSERIAL PRIMARY KEY,
    cliente_id      BIGINT       NOT NULL REFERENCES cartera.cliente(id),
    numero          VARCHAR(40)  NOT NULL,
    serie           VARCHAR(20),
    tipo            VARCHAR(20)  NOT NULL,
```

```
    clave_unicidad   VARCHAR(80)  NOT NULL UNIQUE,
    fecha_emision    DATE         NOT NULL,
    fecha_vencimiento DATE        NOT NULL,
    moneda           VARCHAR(3)   NOT NULL DEFAULT 'USD',
    importe_bruto    NUMERIC(14,2) NOT NULL,
    impuestos        NUMERIC(14,2) NOT NULL DEFAULT 0,
    total            NUMERIC(14,2) NOT NULL,
    saldo_pendiente  NUMERIC(14,2) NOT NULL,
    estado_contable  VARCHAR(20)  NOT NULL,
    fecha_carga      DATE         NOT NULL,
    carga_id         BIGINT       REFERENCES integracion.carga(id),
    created_at       TIMESTAMPTZ  NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CHECK (saldo_pendiente >= 0)
);
CREATE INDEX idx_factura_cliente     ON cartera.factura (cliente_id);
CREATE INDEX idx_factura_vencimiento ON cartera.factura (fecha_vencimiento);
CREATE INDEX idx_factura_estado      ON cartera.factura (estado_contable);
CREATE INDEX idx_factura_saldo_vencido ON cartera.factura (fecha_vencimiento,
saldo_pendiente)
    WHERE saldo_pendiente > 0;

-- Servicio facturado (detalle a nivel de servicios)
CREATE TABLE cartera.servicio_facturado (
    id                   BIGSERIAL PRIMARY KEY,
    factura_id           BIGINT        NOT NULL REFERENCES
cartera.factura(id) ON DELETE CASCADE,
    codigo_servicio      VARCHAR(40)   NOT NULL,
    descripcion          VARCHAR(200),
    periodo              VARCHAR(20),
    cantidad             NUMERIC(12,3) NOT NULL DEFAULT 1,
    precio_unitario      NUMERIC(14,2) NOT NULL,
    importe              NUMERIC(14,2) NOT NULL,
    centro_costo         VARCHAR(40),
    referencia_contratual VARCHAR(60),
    clave_unicidad       VARCHAR(100)  NOT NULL UNIQUE,
    created_at           TIMESTAMPTZ   NOT NULL DEFAULT now(),
    updated_at           TIMESTAMPTZ   NOT NULL DEFAULT now()
);
CREATE INDEX idx_servicio_factura ON cartera.servicio_facturado (factura_id);

-- Pago
CREATE TABLE cartera.pago (
    id                  BIGSERIAL PRIMARY KEY,
    identificador_pago  VARCHAR(60)   NOT NULL UNIQUE,
    cliente_id          BIGINT        NOT NULL REFERENCES cartera.cliente(id),
    fecha               DATE          NOT NULL,
    monto               NUMERIC(14,2) NOT NULL,
    moneda              VARCHAR(3)    NOT NULL DEFAULT 'USD',
    metodo              VARCHAR(30),
    banco               VARCHAR(60),
    referencia_bancaria VARCHAR(80),
    fecha_aplicacion    DATE,
    fecha_carga         DATE          NOT NULL,
    carga_id            BIGINT        REFERENCES integracion.carga(id),
    created_at          TIMESTAMPTZ   NOT NULL DEFAULT now(),
    updated_at          TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CHECK (monto > 0)
);
CREATE INDEX idx_pago_cliente ON cartera.pago (cliente_id);
CREATE INDEX idx_pago_fecha   ON cartera.pago (fecha);

-- Aplicación de pago a factura (N:M)
CREATE TABLE cartera.aplicacion_pago (
    id              BIGSERIAL PRIMARY KEY,
    pago_id         BIGINT        NOT NULL REFERENCES cartera.pago(id) ON
```

```
DELETE CASCADE,
    factura_id       BIGINT        NOT NULL REFERENCES cartera.factura(id),
    monto_aplicado   NUMERIC(14,2) NOT NULL,
    fecha_aplicacion DATE          NOT NULL,
    created_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
    UNIQUE (pago_id, factura_id),
    CHECK (monto_aplicado > 0)
);
CREATE INDEX idx_aplicacion_pago     ON cartera.aplicacion_pago (pago_id);
CREATE INDEX idx_aplicacion_factura  ON cartera.aplicacion_pago (factura_id);

-- Snapshot histórico de cartera (particionado por mes)
CREATE TABLE cartera.snapshot_cartera (
    id             BIGSERIAL,
    fecha_snapshot DATE         NOT NULL,
    cliente_id     BIGINT       NOT NULL REFERENCES cartera.cliente(id),
    saldo_total    NUMERIC(14,2) NOT NULL DEFAULT 0,
    saldo_vencido  NUMERIC(14,2) NOT NULL DEFAULT 0,
    tramo_aging    VARCHAR(10)  NOT NULL,
    nivel_cobranza VARCHAR(4)   NOT NULL,
    PRIMARY KEY (id, fecha_snapshot)
) PARTITION BY RANGE (fecha_snapshot);
```

6. Estrategias Operativas

6.1 Idempotencia y deduplicación

La idempotencia es la garantía operativa más importante del módulo: una misma carga ejecutada dos veces (por reintento, por reproceso o por duplicación del planificador) debe producir exactamente los mismos datos, sin filas duplicadas ni saldos inconsistentes. Se materializa con dos mecanismos combinados. Primero, las restricciones UNIQUE sobre las claves naturales (codigo_erp del cliente, clave_unicidad de factura y servicio, identificador_pago del pago) impiden que existan dos filas para el mismo registro del ERP. Segundo, la operación UPSERT de PostgreSQL (INSERT … ON CONFLICT (clave) DO UPDATE) inserta el registro si no existe o lo actualiza si ya está, en una sola sentencia atómica.

Para evitar escrituras innecesarias y mantener el campo updated_at significativo, la rama DO UPDATE se condiciona con WHERE … IS DISTINCT FROM EXCLUDED, de modo que la fila solo se actualiza si algún dato relevante cambió. Esto reduce la contención y el ruido en la bitácora de auditoría. La sentencia siguiente muestra el patrón para la carga de una factura; el mismo patrón se aplica a clientes, servicios, pagos y aplicaciones.

```
-- Carga idempotente de una factura (INSERT ... ON CONFLICT)
INSERT INTO cartera.factura (
    cliente_id, numero, serie, tipo, clave_unicidad,
    fecha_emision, fecha_vencimiento, moneda,
    importe_bruto, impuestos, total, saldo_pendiente,
    estado_contable, fecha_carga, carga_id
) VALUES (
    :cliente_id, :numero, :serie, :tipo, :clave_unicidad,
    :fecha_emision, :fecha_vencimiento, :moneda,
```

```
    :importe_bruto, :impuestos, :total, :saldo_pendiente,
    :estado_contable, :fecha_carga, :carga_id
)
ON CONFLICT (clave_unicidad) DO UPDATE SET
    saldo_pendiente = EXCLUDED.saldo_pendiente,
    estado_contable = EXCLUDED.estado_contable,
    fecha_carga     = EXCLUDED.fecha_carga,
    carga_id        = EXCLUDED.carga_id,
    updated_at      = now()
WHERE cartera.factura.saldo_pendiente IS DISTINCT FROM
EXCLUDED.saldo_pendiente
   OR cartera.factura.estado_contable IS DISTINCT FROM
EXCLUDED.estado_contable;
```

6.2 Transacción por carga y manejo de errores

Cada carga se ejecuta dentro de una transacción única de PostgreSQL por entidad, de modo que la entidad completa se confirma al final o se revierte entera; nunca queda una entidad a medio cargar. Los registros que no pasan la validación de calidad de datos (RF-INT-04) no abortan la carga: se captura el error a nivel de fila, se inserta su entrada en error_carga con el dato original en JSONB y la carga continúa con el resto del lote. Al final, el estado de la carga queda en ok_errores o fallida según corresponda, y se notifica al administrador. Este patrón de carga tolerante a errores por fila se implementa con SAVEPOINT por registro o con inserciones individuales dentro de la transacción de la entidad.

6.3 Particionamiento de tablas históricas

Las tablas bitacora_integracion y snapshot_cartera crecen de forma indefinida y se consultan típicamente por rango de fechas, por lo que se particionan por rango mensual (RANGE sobre la columna de fecha). El particionamiento declarativo de PostgreSQL 16 permite crear las particiones por adelantado o mediante una función programada que crea la partición del mes siguiente antes de que comience. La consulta por rango de fechas queda confinada a las particiones relevantes (partition pruning), lo que mantiene el tiempo de respuesta estable a medida que crece el histórico.

```
-- Creación de una partición mensual de la bitácora
CREATE TABLE integracion.bitacora_integracion_2026_11
    PARTITION OF integracion.bitacora_integracion
    FOR VALUES FROM ('2026-11-01') TO ('2026-12-01');

-- Creación de una partición mensual del snapshot
CREATE TABLE cartera.snapshot_cartera_2026_11
    PARTITION OF cartera.snapshot_cartera
    FOR VALUES FROM ('2026-11-01') TO ('2026-12-01');

-- Partición por defecto para fechas fuera de rango (evita inserciones
fallidas)
CREATE TABLE integracion.bitacora_integracion_default
    PARTITION OF integracion.bitacora_integracion DEFAULT;
CREATE TABLE cartera.snapshot_cartera_default
    PARTITION OF cartera.snapshot_cartera DEFAULT;
```

Se recomienda mantener un procedimiento programado (job mensual) que cree la partición del mes siguiente antes del día uno, de modo que las inserciones nunca caigan en la partición por defecto excepto ante una falla del job, situación que debe alertarse.

6.4 Retención y purga

La bitácora de integración y los errores resueltos se conservan durante el período definido por la política de auditoría (recomendado: dos años para la bitácora y un año para los errores resueltos). La purga se realiza por partición completa mediante DETACH PARTITION y DROP TABLE, operación que no bloquea la inserción concurrente y es instantánea independientemente del volumen. Los snapshots de cartera se conservan indefinidamente por su valor histórico; si la política lo exige, pueden archivarse a una tabla externa o exportarse antes de su purga.

```
-- Purga de una partición mensual vencida (instantánea, sin bloqueo)
ALTER TABLE integracion.bitacora_integracion
    DETACH PARTITION integracion.bitacora_integracion_2024_11;
DROP TABLE integracion.bitacora_integracion_2024_11;
```

6.5 Vistas de conveniencia

Para facilitar las consultas del módulo de Cartera y Reportes sin exponer la complejidad del modelo, se definen vistas de solo lectura sobre las tablas cargadas. Estas vistas pertenecen al esquema cartera y se conceden a cxc_app y cxc_readonly.

```
-- Vista de cartera activa por cliente
CREATE OR REPLACE VIEW cartera.v_cartera_cliente AS
SELECT c.id              AS cliente_id,
       c.codigo_erp,
       c.razon_social,
       COUNT(f.id)       AS facturas_abiertas,
       COALESCE(SUM(f.saldo_pendiente), 0) AS saldo_total,
       COALESCE(SUM(f.saldo_pendiente)
                FILTER (WHERE f.fecha_vencimiento < CURRENT_DATE), 0) AS
saldo_vencido
FROM   cartera.cliente c
LEFT JOIN cartera.factura f
       ON f.cliente_id = c.id AND f.saldo_pendiente > 0
WHERE  c.activo = TRUE
GROUP BY c.id, c.codigo_erp, c.razon_social;
GRANT SELECT ON cartera.v_cartera_cliente TO cxc_app, cxc_readonly;
```

7. Seguridad a Nivel de Base de Datos

7.1 Roles y mínimo privilegio

La base de datos se opera con roles separados por propósito, de modo que ningún rol tenga más privilegios de los estrictamente necesarios. La aplicación web (cxc_app) solo puede seleccionar, insertar y actualizar en las tablas de negocio; no tiene permisos de eliminación,

en cumplimiento de la inmutabilidad operativa (RNF-SEG-06). El proceso de carga (cxc_loader) tiene los permisos de escritura necesarios en integracion y cartera, incluida la inserción en la bitácora. El rol de solo lectura (cxc_readonly) cubre los reportes y la auditoría. La propiedad administrativa (cxc_owner) no se usa desde la aplicación.

Tabla 19. Matriz de privilegios por rol y esquema

Rol	Esquema integracion	Esquema cartera
cxc_app (aplicación web)	SELECT, INSERT, UPDATE	SELECT, INSERT, UPDATE
cxc_loader (carga batch)	SELECT, INSERT, UPDATE, DELETE	SELECT, INSERT, UPDATE
cxc_readonly (reportes)	SELECT	SELECT
cxc_owner (administración)	Todos (DDL + DML)	Todos (DDL + DML)

7.2 Protección de credenciales del ERP

Las credenciales de acceso al ERP se consideran secretos y nunca se almacenan en claro en la base de datos ni en el código. La tabla conexion_erp guarda únicamente una referencia (credenciales_ref) al secreto gestionado por la capa de aplicación (almacén de secretos del entorno o servicio de cifrado), de modo que la base de datos nunca contiene el valor usable; para el API REST, dicho secreto es el token de portador o el par cliente-servidor OAuth2 que el adaptador presenta en cada petición. Los logs y la bitácora se filtran para impedir que un secreto llegue a registrarse; a tal efecto, el conector marca los campos sensibles y el logger los enmascara antes de cualquier registro.

7.3 Auditoría de acceso

PostgreSQL mantiene su propio log de conexiones y consultas lentas (log_connections, log_min_duration_statement) configurable por el administrador. Complementariamente, la bitácora de integración registra todos los eventos relevantes del proceso de carga con su contexto, y la tabla de errores conserva el dato original de cada registro rechazado para su análisis. Para las acciones del módulo de Cobranza (no cubierto por este documento) la auditoría a nivel de aplicación registrará usuario, operación y valores antes y después, conforme al requisito RNF-SEG-03 del SRS.

8. Migraciones y Versionado del Esquema

8.1 Herramienta de migraciones

El esquema se gestiona con migraciones versionadas nativas de Laravel, integradas en el ciclo de despliegue de la aplicación. Cada migración es una clase PHP con dos métodos (up y

down) que contienen el DDL correspondiente, ejecutado mediante DB::statement dado que es SQL estándar de PostgreSQL; el orden de aplicación se controla por la marca de tiempo del nombre del archivo. Las migraciones se ejecutan automáticamente en el despliegue (php artisan migrate --force) y su estado queda registrado en la tabla migrations de control, de modo que cada entorno sabe con exactitud qué versión del esquema tiene aplicada.

8.2 Convenciones de las migraciones

- Una migración por cambio lógico del esquema, no una por tabla; el DDL completo de la Etapa 1 se agrupa en una migración inicial.
- Toda migración debe ser idempotente en lo posible (usar IF NOT EXISTS / IF EXISTS) para tolerar re-ejecuciones parciales.
- Las migraciones destructivas (DROP de columnas o tablas) se dividen en dos: primero dejar de usar el elemento (una versión), y tras un ciclo de validación, eliminarlo (la siguiente versión).
- Las migraciones que cargan datos de catálogo (condiciones de pago, parámetros) usan INSERT … ON CONFLICT DO NOTHING para no duplicar.
- Toda migración se prueba en un entorno espejo antes de aplicarse a producción, y se respalda la base de datos inmediatamente antes.

8.3 Compatibilidad hacia atrás

Para permitir despliegues sin tiempo de incautación y rollbacks seguros, las migraciones mantienen la compatibilidad hacia atrás durante un ciclo. Una nueva columna se añade anulable o con valor por defecto, se popula en segundo plano, y solo tras confirmar que la versión anterior de la aplicación ya no corre se aplica la restricción NOT NULL o se elimina la columna antigua. Este patrón evita bloqueos prolongados sobre las tablas de mayor volumen (factura, servicio_facturado) durante las ventanas de mantenimiento.

9. Volumetría Estimada y Trazabilidad

9.1 Volumetría estimada y crecimiento

La tabla 20 estima el volumen anual de cada tabla a partir de una operación de referencia (10 000 clientes activos, 120 000 facturas anuales con un promedio de 4 servicios cada una y 100 000 pagos anuales). Las estimaciones sirven para dimensionar los espacios y anticipar las políticas de particionamiento y purga. Los números son de referencia y se ajustarán con los datos reales de la organización.

Tabla 20. Volumetría estimada anual

Tabla	Filas/año (est.)	Crecimiento	Estrategia
cliente	~2 000 deltas	Lento	Borrado lógico; sin partición.
condicion_pago	<100 deltas	Muy lento	Catálogo; sin partición.
factura	~120 000	Lineal	Índices selectivos; sin partición inicial.
servicio_facturado	~480 000	Lineal	Índices por factura; sin partición inicial.
pago	~100 000	Lineal	Índices por cliente y fecha.
aplicacion_pago	~150 000	Lineal	Índices por pago y factura.
carga	~365 (diaria)	Lento	Sin partición.
bitacora_integracion	~500 000+	Rápido	Particionada mensual; purga a 2 años.
snapshot_cartera	~3,6 M (10k×365)	Rápido	Particionada mensual; retención indefinida o por política.
error_carga	Variable	Variable	Purga de resueltos a 1 año.

9.2 Trazabilidad de requisitos a tablas

La tabla 21 vincula los requisitos funcionales del SRS con las tablas que los materializan, de modo que la revisión de cobertura de la Etapa 1 pueda verificar que ningún requisito del módulo quede sin soporte de datos.

Tabla 21. Trazabilidad requisito → tablas

Requisito (SRS)	Tablas que lo soportan
RF-INT-01 Carga batch diaria	carga, carga_detalle, bitacora_integracion
RF-INT-02 Detalle a nivel de servicios	factura, servicio_facturado
RF-INT-03 Idempotencia y deduplicación	cliente, factura, servicio_facturado, pago (UQ + ON CONFLICT)
RF-INT-04 Validación de calidad de datos	error_carga (cola de rechazos), CHECK constraints
RF-INT-05 Bitácora de cargas	carga, carga_detalle, bitacora_integracion
RF-INT-06 Alertas de falla	carga.estado, bitacora_integracion.nivel
RF-INT-07 Consulta dirigida por API	carga (origen=api_dirigida), factura, pago
RF-INT-08 Reproceso controlado	carga (origen=reproceso), carga.usuario_reproceso
RF-INT-09 Histórico de cartera	snapshot_cartera (particionada)

Requisito (SRS)	Tablas que lo soportan
RF-INT-10 Detección de cambios de maestros	cliente, condicion_pago (updated_at, fecha_baja)
RNF-SEG-06 Inmutabilidad operativa	Borrado lógico (activo, fecha_baja); sin DELETE desde la app
RNF-SEG-03 Auditoría	created_at, updated_at, updated_by; bitacora_integracion

Este documento, junto con el DDL aquí presentado y el diagrama conceptual de la figura 1, constituye la base de implementación de la base de datos de la Etapa 1. Las decisiones pendientes (mapeo físico real de campos del ERP, moneda y zona horaria operativas, período de retención de auditoría) se completarán al inicio de la etapa con la documentación técnica del sistema contable de la organización.